

**UNIVERSIDADE VEIGA DE ALMEIDA
CAMPUS TIJUCA
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

LUCAS PIRES DA COSTA
RUAN DA SILVA MARQUES
LUCAS SANTOS GARCIA

MONTA AÍ:

sistema especialista que recomenda computadores

RIO DE JANEIRO
2026

LUCAS PIRES DA COSTA – 1250122691
RUAN DA SILVA MARQUES – 1250124415
LUCAS SANTOS GARCIA – 1250117441

MONTA AÍ:

sistema especialista que recomenda computadores

Trabalho apresentado à disciplina Programação de Sistemas Especialistas, do curso de Ciência da Computação da Universidade Veiga de Almeida, Campus Tijuca, como requisito parcial para aprovação na Atividade Avaliativa A1.

Professor: Vinicius Marques da Silva Ferreira.

RIO DE JANEIRO

2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 FUNDAMENTAÇÃO TEÓRICA.....	4
2.1 Sistemas especialistas.....	4
2.2 Componentes.....	4
2.3 Encadeamento para frente.....	4
2.4 Prolog e Python.....	5
3 MINIMUNDO.....	6
4 FLUXOGRAMA DO SISTEMA ESPECIALISTA.....	7
5 BASE DE CONHECIMENTO E REGRAS.....	9
5.1 Base de fatos.....	9
5.2 Regras.....	9
5.3 Exemplo de encadeamento.....	10
6 DIAGRAMA DE CASO DE USO.....	12
7 DIAGRAMA DE ESTADO.....	13
8 IMPLEMENTAÇÃO EM PYTHON.....	14
8.1 Representação das regras.....	14
8.2 Motor de inferência.....	14
9 PROTÓTIPO DE TELAS.....	16
10 CONSIDERAÇÕES FINAIS.....	18
REFERÊNCIAS.....	19

1 INTRODUÇÃO

Escolher as peças de um computador é difícil para quem não acompanha o mercado de hardware. Quem vai comprar precisa decidir qual processador, quanta memória, se precisa ou não de placa de vídeo e quanto armazenamento, sem gastar mais do que pode. Na prática, essa decisão costuma ser feita perguntando a alguém que entende do assunto: um especialista.

Este trabalho apresenta o **Monta Aí**, um sistema especialista que faz o papel desse especialista. O usuário responde a três perguntas no terminal (uso do computador, orçamento e formato) e o sistema, por meio de uma base de 20 regras, recomenda as peças, emite avisos e explica quais regras usou para chegar à resposta.

A área foi escolhida porque os integrantes do grupo, estudantes de Ciência da Computação, montam e configuram os próprios computadores e são frequentemente consultados sobre o que comprar. O trabalho foi desenvolvido na disciplina Programação de Sistemas Especialistas, e o objetivo é construir um sistema especialista simples em Python, com base de conhecimento, base de fatos, motor de inferência por encadeamento para frente e explicação, além do fluxograma, dos diagramas e do protótipo de telas.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Sistemas especialistas

Sistemas especialistas são programas que resolvem problemas de uma área restrita usando o conhecimento de especialistas humanos, representado de forma que o computador consiga raciocinar com ele (GIARRATANO; RILEY, 2005). A principal característica é a separação entre o conhecimento, guardado em uma base de regras, e o mecanismo que o utiliza, o que permite alterar o comportamento do sistema apenas editando as regras (LUGER, 2013; FERREIRA, 2026).

2.2 Componentes

- **base de conhecimento:** regras no formato SE condições ENTÃO conclusões;
- **base de fatos:** o que se sabe sobre o caso: as respostas do usuário e o que for deduzido;
- **motor de inferência:** compara as regras com os fatos e aplica as que forem verdadeiras;
- **explicação:** mostra ao usuário como a conclusão foi alcançada;
- **interface:** faz as perguntas e mostra o resultado.

2.3 Encadeamento para frente

No encadeamento para frente, o raciocínio parte dos fatos conhecidos em direção às conclusões: o motor aplica as regras cujas condições são verdadeiras, acrescenta as conclusões à base de fatos e repete o processo até que nenhuma regra nova possa ser aplicada (RUSSELL; NORVIG, 2022). Como os fatos novos podem ativar outras regras, as conclusões vão se encadeando.

2.4 Prolog e Python

Sistemas especialistas são tradicionalmente escritos em Prolog, linguagem em que o conhecimento é descrito como fatos e regras e a própria linguagem faz a inferência (BRATKO, 2011; BRAMER, 2005; BLACKBURN; BOS; STRIEGNITZ, 2014). Neste trabalho, como pede a atividade, foi usado Python, em que o motor de inferência é escrito explicitamente. As estruturas nativas da linguagem, como listas e dicionários, tornam simples representar os fatos e as regras (BORGES, 2014; BANIN, 2018; COSTA, 2015; PAIVA, 2021). Python também é a linguagem mais usada em ciência de dados e inteligência artificial (KLOSTERMAN, 2020; NETTO; MACIEL, 2021; ANGELOV; SCAVETTA, 2022).

3 MINIMUNDO

O Monta Aí atende pessoas que querem comprar um computador e não sabem quais peças escolher. O sistema roda no terminal e faz três perguntas ao usuário: para que ele vai usar o computador (estudo, programação, jogos ou edição de vídeo), quanto pode gastar (orçamento baixo, até R\$ 3.000; médio, de R\$ 3.000 a R\$ 6.000; ou alto, acima de R\$ 6.000) e se prefere um desktop ou um notebook.

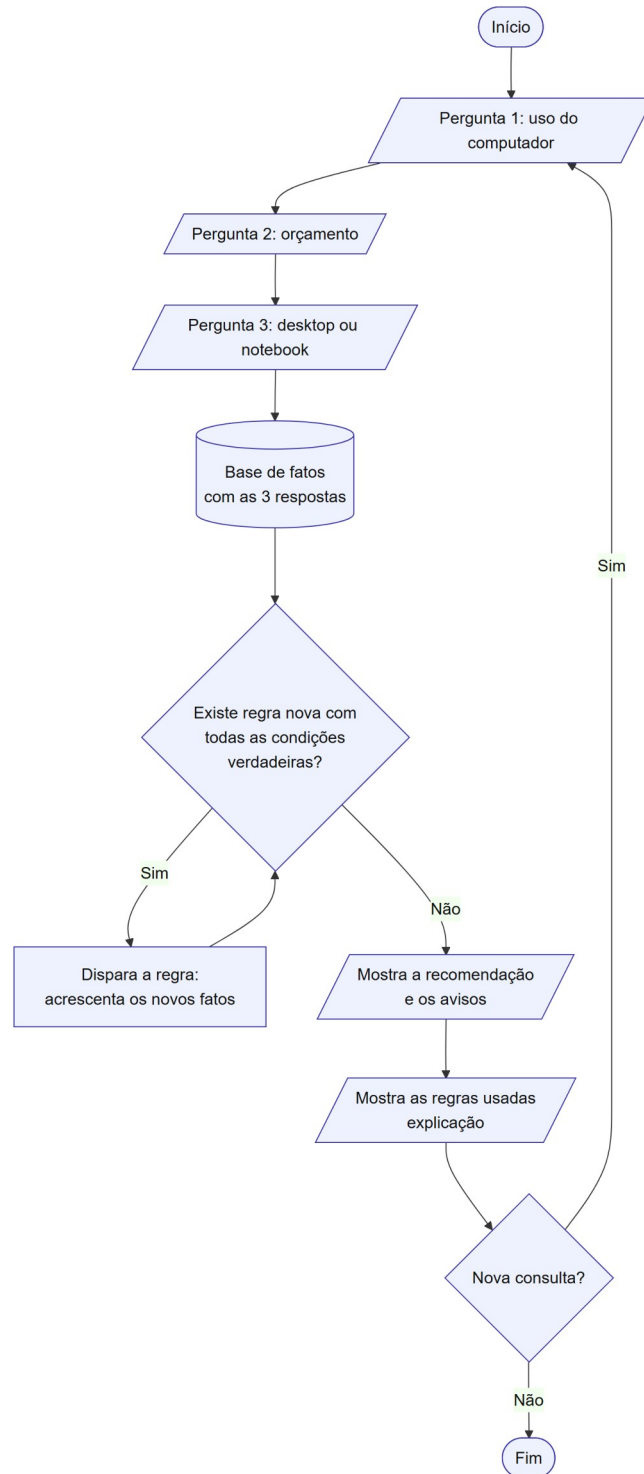
Pelo uso, o sistema identifica o perfil do usuário: básico, desenvolvedor, gamer ou criador de conteúdo. Cada perfil define o processador, a memória RAM e se o computador precisa de placa de vídeo dedicada. O orçamento define o tamanho do SSD e, quando há placa de vídeo dedicada, o nível dessa placa. O formato gera dicas: no notebook não é possível trocar peças depois, enquanto no desktop é possível melhorar aos poucos.

O sistema também avisa quando o orçamento está apertado para jogos ou edição de vídeo e quando um notebook com placa de vídeo dedicada precisa de boa refrigeração. No final, mostra a recomendação e a lista das regras usadas, na ordem em que foram aplicadas, para que o usuário entenda o porquê de cada escolha. O especialista em hardware é quem cadastra e altera as regras.

4 FLUXOGRAMA DO SISTEMA ESPECIALISTA

A Figura 1 mostra o fluxo do sistema. As três respostas formam a base de fatos. Em seguida, o motor de inferência verifica se existe alguma regra nova com todas as condições verdadeiras; se existir, a regra é disparada e seus fatos são acrescentados, e a verificação se repete. Quando nenhuma regra nova pode ser aplicada, o sistema mostra a recomendação e as regras usadas, e pergunta se o usuário quer fazer uma nova consulta.

Figura 1 – Fluxograma do Monta Aí



Fonte: elaborado pelos autores (2026).

5 BASE DE CONHECIMENTO E REGRAS

5.1 Base de fatos

A base de fatos é um dicionário que começa com as três respostas do usuário (Quadro 1) e recebe os fatos deduzidos pelas regras (Quadro 2).

Quadro 1 – Fatos iniciais (respostas do usuário)

Fato	Pergunta	Valores possíveis
uso	Para que você vai usar o computador?	estudo, programação, jogos, edição de vídeo
orcamento	Quanto você pode gastar?	baixo, médio, alto
formato	Você prefere desktop ou notebook?	desktop, notebook

Fonte: elaborado pelos autores (2026).

Quadro 2 – Fatos deduzidos pelas regras

Fato	Significado	Deduzido por
perfil	Perfil do usuário	R1 a R4
processador, memoria, video	Peças do perfil e tipo de vídeo (integrado ou dedicado)	R5 a R8
armazenamento	Tamanho do SSD	R9 a R11
placa_de_video	Placa de vídeo recomendada	R12 a R15
aviso_orcamento, aviso_notebook	Avisos ao usuário	R16 a R18
dica	Dica sobre o formato	R19 e R20

Fonte: elaborado pelos autores (2026).

5.2 Regras

A base de conhecimento tem 20 regras, organizadas em cinco grupos: perfil (R1 a R4), peças do perfil (R5 a R8), armazenamento (R9 a R11), placa de vídeo (R12 a R15) e avisos e dicas (R16 a R20). O Quadro 3 apresenta todas elas.

Quadro 3 – Base de conhecimento do Monta Ai

Regra	SE	ENTÃO
R1	uso = estudo	perfil = básico

Regra	SE	ENTÃO
R2	uso = programação	perfil = desenvolvedor
R3	uso = jogos	perfil = gamer
R4	uso = edição de vídeo	perfil = criador de conteúdo
R5	perfil = básico	processador = de entrada (4 núcleos) E memória = 8 GB E vídeo = integrado
R6	perfil = desenvolvedor	processador = intermediário (6 núcleos) E memória = 16 GB E vídeo = integrado
R7	perfil = gamer	processador = intermediário (6 núcleos) E memória = 16 GB E vídeo = dedicado
R8	perfil = criador de conteúdo	processador = avançado (8 núcleos ou mais) E memória = 32 GB E vídeo = dedicado
R9	orcamento = baixo	armazenamento = SSD de 256 GB
R10	orcamento = médio	armazenamento = SSD de 512 GB
R11	orcamento = alto	armazenamento = SSD de 1 TB
R12	video = integrado	placa_de_video = não precisa (usa o vídeo do processador)
R13	video = dedicado E orcamento = baixo	placa_de_video = placa de entrada
R14	video = dedicado E orcamento = médio	placa_de_video = placa intermediária
R15	video = dedicado E orcamento = alto	placa_de_video = placa avançada
R16	perfil = gamer E orcamento = baixo	aviso_orcamento = Orçamento apertado para jogos: dá para jogar, mas com gráficos no mínimo.
R17	perfil = criador de conteúdo E orcamento = baixo	aviso_orcamento = Orçamento apertado para edição: os vídeos vão demorar para renderizar.
R18	formato = notebook E video = dedicado	aviso_notebook = Notebook com placa de vídeo esquentada: escolha um modelo com boa refrigeração.
R19	formato = notebook	dica = No notebook não dá para trocar processador nem placa: compre pensando no futuro.
R20	formato = desktop	dica = No desktop dá para trocar peças depois: comece simples e melhore aos poucos.

Fonte: elaborado pelos autores (2026).

5.3 Exemplo de encadeamento

Considere um usuário que vai usar o computador para jogos, tem orçamento médio e quer um notebook. O Quadro 4 mostra as regras aplicadas pelo motor, na ordem.

Quadro 4 – Regras aplicadas no exemplo

Ordem	Regra	Fatos acrescentados
1	R3	perfil = gamer
2	R7	processador = intermediário (6 núcleos) E memória = 16 GB E vídeo = dedicado

Ordem	Regra	Fatos acrescentados
3	R10	armazenamento = SSD de 512 GB
4	R14	placa_de_video = placa intermediária
5	R18	aviso_notebook = Notebook com placa de vídeo esquentada: escolha um modelo com boa refrigeração.
6	R19	dica = No notebook não dá para trocar processador nem placa: compre pensando no futuro.

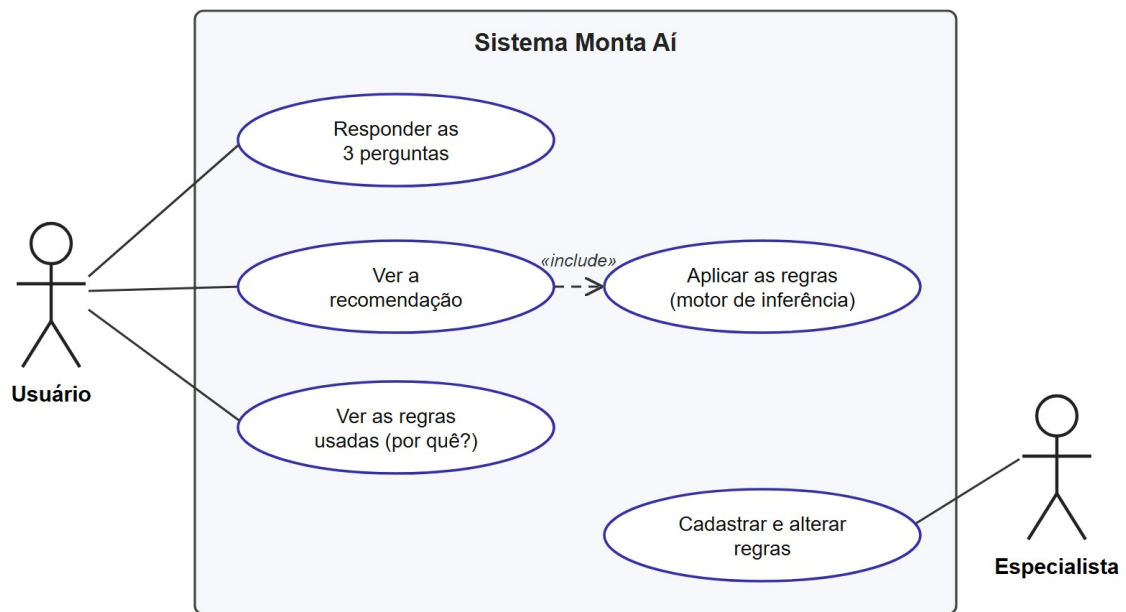
Fonte: elaborado pelos autores (2026).

O encadeamento aparece em três níveis: a R3 deduz o perfil gamer; a R7 usa esse perfil para deduzir que o computador precisa de vídeo dedicado; e a R14 usa o vídeo dedicado, junto com o orçamento médio, para recomendar uma placa intermediária. O usuário não informou nenhum desses fatos: todos foram deduzidos pelas regras.

6 DIAGRAMA DE CASO DE USO

O diagrama de caso de uso (Figura 2) tem dois atores. O **Usuário** responde às três perguntas, vê a recomendação e vê as regras usadas. Ver a recomendação inclui a aplicação das regras pelo motor de inferência. O **Especialista** cadastra e altera as regras da base de conhecimento.

Figura 2 – Diagrama de caso de uso do Monta Aí

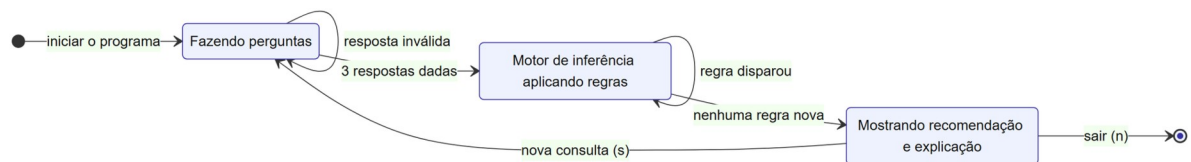


Fonte: elaborado pelos autores (2026).

7 DIAGRAMA DE ESTADO

O diagrama de estado (Figura 3) mostra os três estados do sistema: fazendo perguntas, em que permanece enquanto a resposta for inválida; aplicando regras, em que permanece enquanto alguma regra disparar; e mostrando a recomendação e a explicação. A partir do resultado, o usuário pode fazer uma nova consulta, voltando às perguntas, ou sair.

Figura 3 – Diagrama de estado do Monta Aí



Fonte: elaborado pelos autores (2026).

8 IMPLEMENTAÇÃO EM PYTHON

Todo o sistema está em um único arquivo, `monta_ai.py`, escrito em Python 3 sem bibliotecas externas e comentado parte por parte. O código está disponível no GitHub:

Repositório: <https://github.com/BrasDP/monta-ai>

Quadro 5 – Partes do arquivo `monta_ai.py`

Parte	Onde está no código
Minimundo	Comentário no início do arquivo
Base de conhecimento	Lista REGRAS
Base de fatos	Dicionário fatos
Motor de inferência	Funções <code>condicoes_verdadeiras</code> e <code>motor_de_inferencia</code>
Explicação	Função <code>mostrar_explicacao</code>
Interface	Funções <code>perguntar</code> , <code>coletar_fatos</code> e <code>mostrar_recomendacao</code>

Fonte: elaborado pelos autores (2026).

8.1 Representação das regras

Cada regra é um dicionário com um identificador, as condições (“se”) e os fatos que acrescenta (“entao”). Exemplos:

```
{ "id": "R3", "se": { "uso": "jogos" }, "entao": { "perfil": "gamer" } },
{ "id": "R14", "se": { "video": "dedicado", "orcamento": "médio" },
  "entao": { "placa_de_video": "placa intermediária" } },
```

8.2 Motor de inferência

O motor percorre as regras e dispara as que têm todas as condições verdadeiras, acrescentando seus fatos. Cada regra dispara no máximo uma vez. O laço se repete enquanto alguma regra disparar, porque os fatos novos podem ativar outras regras; quando uma volta termina sem disparos, o motor para. A lista de regras usadas é devolvida para a explicação.

```

def condicoes_verdadeiras(regra, fatos):
    """Retorna True se TODAS as condições da regra estão na base de fatos."""
    for nome, valor in regra["se"].items():
        if fatos.get(nome) != valor:
            return False
    return True

def motor_de_inferencia(fatos):
    """Aplica as regras sobre os fatos e devolve a lista de regras usadas, na ordem."""
    regras_usadas = []
    alguma_regra_disparou = True

    while alguma_regra_disparou:          # repete enquanto surgirem fatos novos
        alguma_regra_disparou = False
        for regra in REGRAS:
            if regra["id"] in regras_usadas:
                continue                # essa regra já foi usada, pula
            if condicoes_verdadeiras(regra, fatos):
                fatos.update(regra["entao"])    # acrescenta os novos fatos
                regras_usadas.append(regra["id"]) # guarda para a explicação
                alguma_regra_disparou = True

    return regras_usadas

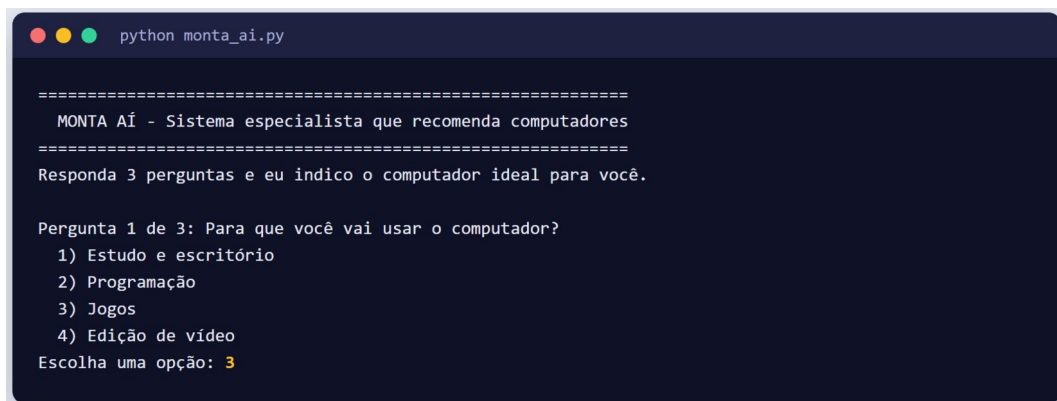
```


9 PROTÓTIPO DE TELAS

Como o sistema roda no terminal, as telas são as do próprio programa. O protótipo, com a saída real do `monta_ai.py`, está publicado no GitHub Pages no endereço abaixo, e as Figuras 4 a 7 mostram suas telas no exemplo da seção 5.3. Em amarelo, o que o usuário digitou.

Protótipo: <https://brasdp.github.io/monta-ai/prototipo/>

Figura 4 – Tela 1: início e primeira pergunta



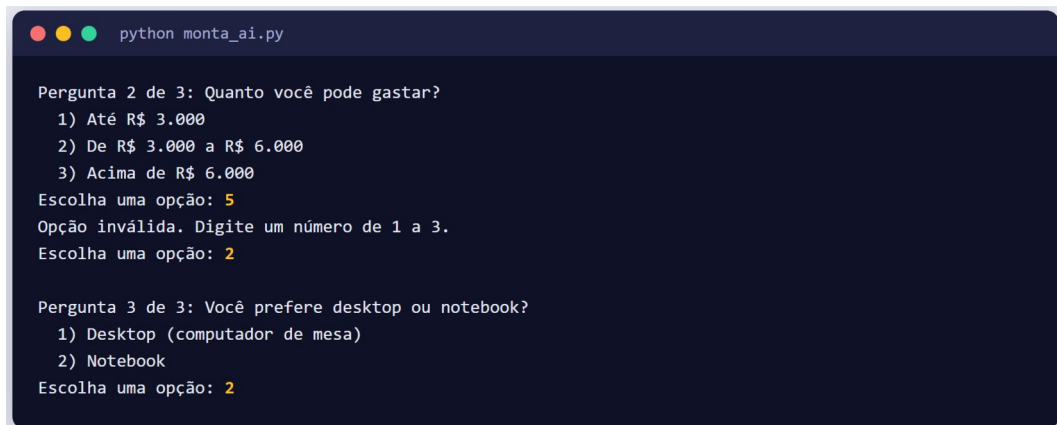
```
python monta_ai.py

=====
MONTA AÍ - Sistema especialista que recomenda computadores
=====
Responda 3 perguntas e eu indico o computador ideal para você.

Pergunta 1 de 3: Para que você vai usar o computador?
1) Estudo e escritório
2) Programação
3) Jogos
4) Edição de vídeo
Escolha uma opção: 3
```

Fonte: elaborado pelos autores (2026).

Figura 5 – Tela 2: segunda e terceira perguntas, com uma opção inválida



```
python monta_ai.py

Pergunta 2 de 3: Quanto você pode gastar?
1) Até R$ 3.000
2) De R$ 3.000 a R$ 6.000
3) Acima de R$ 6.000
Escolha uma opção: 5
Opção inválida. Digite um número de 1 a 3.
Escolha uma opção: 2

Pergunta 3 de 3: Você prefere desktop ou notebook?
1) Desktop (computador de mesa)
2) Notebook
Escolha uma opção: 2
```

Fonte: elaborado pelos autores (2026).

Figura 6 – Tela 3: recomendação

```
python monta_ai.py

-----
RECOMENDAÇÃO
-----

Perfil identificado: gamer
Processador:      intermediário (6 núcleos)
Memória RAM:      16 GB
Placa de vídeo:   placa intermediária
Armazenamento:   SSD de 512 GB

ATENÇÃO: Notebook com placa de vídeo esquentada: escolha um modelo com boa refrigeração.

DICA: No notebook não dá para trocar processador nem placa: compre pensando no futuro.
```

Fonte: elaborado pelos autores (2026).

Figura 7 – Tela 4: explicação com as regras usadas

```
python monta_ai.py

-----
POR QUÊ? Regras usadas, na ordem em que o motor aplicou:
-----

R3: SE uso = jogos ENTÃO perfil = gamer
R7: SE perfil = gamer ENTÃO processador = intermediário (6 núcleos) E memória = 16 GB E vídeo =
dedicado
R10: SE orçamento = médio ENTÃO armazenamento = SSD de 512 GB
R14: SE vídeo = dedicado E orçamento = médio ENTÃO placa_de_vídeo = placa intermediária
R18: SE formato = notebook E vídeo = dedicado ENTÃO aviso_notebook = Notebook com placa de vídeo
esquentada: escolha um modelo com boa refrigeração.
R19: SE formato = notebook ENTÃO dica = No notebook não dá para trocar processador nem placa:
compre pensando no futuro.

Deseja fazer uma nova consulta? (s/n): n
Até mais!
```

Fonte: elaborado pelos autores (2026).

10 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o Monta Aí, um sistema especialista simples que recomenda computadores. Com três perguntas e 20 regras, o sistema mostra os componentes de um sistema especialista: a base de conhecimento separada do motor de inferência, a base de fatos que cresce durante o raciocínio, o encadeamento para frente e a explicação das regras usadas.

Como limitações, o sistema recomenda categorias de peças e não modelos ou preços, e considera apenas três perguntas. Como trabalhos futuros, podem ser acrescentadas perguntas (como a resolução do monitor) e novas regras, sem alterar o motor de inferência, e uma interface gráfica.

REFERÊNCIAS

ANGELOV, Boyan; SCAVETTA, Rick J. **Python e R para o cientista de dados moderno**: o melhor de dois mundos. São Paulo: Novatec, 2022.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BANIN, Sérgio Luiz. **Python 3**: conceitos e aplicações: uma abordagem didática. São Paulo: Érica, 2018.

BLACKBURN, Patrick; BOS, Johan; STRIEGNITZ, Kristina. **Aprenda Prolog já!** New York: College Publications, 2014.

BORGES, Luiz Eduardo. **Python para desenvolvedores**. São Paulo: Novatec, 2014.

BRAMER, Max. **Logic programming with Prolog**. New York: Springer, 2005.

BRATKO, Ivan. **Prolog programming for artificial intelligence**. New York: Addison-Wesley, 2011.

COSTA, Ernesto. **Programação em Python**: fundamentos e resolução de problemas. São Paulo: FCA Editora, 2015.

COSTA, Lucas Pires da; MARQUES, Ruan da Silva; GARCIA, Lucas Santos. **Monta Aí**: sistema especialista que recomenda computadores. [S. l.]: GitHub, 2026. Disponível em: <https://github.com/BrasDP/monta-ai>. Acesso em: 26 set. 2026.

FERREIRA, Vinicius Marques da Silva. **Programação de Sistemas Especialistas**. Rio de Janeiro: Universidade Veiga de Almeida, 2026. Slides de aula.

GIARRATANO, Joseph C.; RILEY, Gary D. **Expert systems**: principles and programming. 4. ed. Boston: Thomson Course Technology, 2005.

KLOSTERMAN, Stephen. **Projetos de ciência de dados com Python**: abordagem de estudo de caso para criação de projetos de ciência de dados bem-sucedidos usando Python, pandas e scikit-learn. São Paulo: Novatec, 2020.

LUGER, George F. **Inteligência artificial**. 6. ed. São Paulo: Pearson Education do Brasil, 2013.

NETTO, Amilcar; MACIEL, Francisco. **Python para data science**: e machine learning descomplicado. Rio de Janeiro: Alta Books, 2021.

PAIVA, Severino. **Introdução à programação e ao pensamento computacional**: usando linguagem Python. Rio de Janeiro: Ciência Moderna, 2021.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência artificial**: uma abordagem moderna. 4. ed. Rio de Janeiro: GEN LTC, 2022.